2026-11-04 NextAuth OAuth 설정과 연결

- 날짜: 2026-11-04
- 앱: `lgcns-fe-next`
- 패키지: `next` 16.4.3, `react` 19.3.0, `next-auth` ^4.24.15 (`lgcns-fe-next/package.json`)
- 기준: 이 문서의 "현재 코드"는 지금 디스크의 파일이다. 환경 변수 값, 시크릿, 토큰 문자열은 적지 않는다.

이 앱의 소스에는 `<script>` 태그가 없다. Next가 TypeScript 모듈을 묶는다. `"use client"`와 라우트 핸들러의 `export { GET, POST }`는 HTML의 `async`, `defer`, `type="module"`이 아니다.

1. 파일이 언제 실행되는가

파일	선언	실행 시점	HTML script 와의 차이
`lgcns-fe-next/app/api/auth/[...nextauth]/route.ts`	`const nextAuthHandler = NextAuth(authOptions)` 다음 `export { nextAuthHandler as GET, nextAuthHandler as POST }`	브라우저가 `/api/auth/...`로 GET 또는 POST를 보낼 때 서버가 그 함수를 호출한다	페이지 HTML에 붙지 않는다. `async/defer`로 미루지 않는다. 요청이 와야 실행된다
`lgcns-fe-next/lib/authOptions.ts`	`export const authOptions`	위 라우트와 `AuthShell`이 이 모듈을 import할 때 서버에서 평가된다	브라우저 스크립트가 아니다. 클라이언트 번들에 넣지 않는다
`lgcns-fe-next/types/next-auth.d.ts`	`declare module "next-auth"`	컴파일할 때 타입만 합친다. 런타임 코드가 없다	로드되지 않는다. `type="module"` 스크립트가 아니다
`lgcns-fe-next/component/molecules/GoogleLoginButton.tsx`	파일 첫 줄 `"use client"`	로그인 페이지 첫 HTML을 서버가 그린 뒤, 브라우저가 컴포넌트를 hydrate한다. `signIn("google")`은 클릭 이후에만 실행된다	클릭 핸들러는 모듈이 평가될 때가 아니라 이벤트 때 돈다
`lgcns-fe-next/component/molecules/KakaoLoginButton.tsx`	파일 첫 줄 `"use client"`	함수 `signIn("kakao")`	마운트되지 않으면 클릭도 없

파일	선언	실행 시점	HTML script 와의 차이
		는 파일에 있다. `LoginSection`이 이 컴포넌트를 주석으로 감싸 두어, 현재 로그인 화면에는 마운트되지 않는다	고 네트워크 요청도 없다
`lgcns-fe-next/component/providers/AuthShell.tsx`	`export default async function`	루트 레이아웃의 `<Suspense>` 안에서 요청마다 서버가 실행한다. 앞에 `await connection()`이 있다	브라우저 모듈이 아니다

HTML에서 `async`는 스크립트를 받는 즉시 실행하고 문서 파싱을 막지 않는다. `defer`는 문서를 파싱한 뒤, `DOMContentLoaded` 전에 순서대로 실행한다. `type="module"`은 지연 실행되는 ES 모듈이다. 이 앱의 OAuth 시작점은 그 속성이 아니라 서버 라우트와 클라이언트 클릭이다.

참고: HTML script 요소 https://html.spec.whatwg.org/multipage/scripting.html#the-script-element, MDN script https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script, Next.js Route Handlers https://nextjs.org/docs/app/building-your-application/routing/route-handlers, TypeScript declaration merging https://www.typescriptlang.org/docs/handbook/declaration-merging.html

2. 공급자 등록

`authOptions.providers`에 세 공급자가 있다.

id	만드는 함수	클라이언트에서 부르는 이름	현재 화면
`credentials`	`CredentialsProvider`	`signIn("credentials", { redirect: false })` (`LoginForm.tsx`)	로그인 폼이 렌더된다
`kakao`	`KakaoProvider`	`signIn("kakao")` (`KakaoLoginButton.tsx`)	`LoginSection.tsx`에서 주석으로 빠져 있다
`google`	`GoogleProvider`	`signIn("google")` (`GoogleLoginButton.tsx`)	`LoginSection`이 이 버튼을 렌더한다

설치 패키지의 카카오 공급자(`node_modules/next-auth/providers/kakao.js`)는 다음 주소를 쓴다.

- 인가: `https://kauth.kakao.com/oauth/authorize?scope`
- 토큰: `https://kauth.kakao.com/oauth/token`
- 사용자: `https://kapi.kakao.com/v2/user/me`

인가 주소의 scope 쿼리는 패키지 문자열에만 있고 값이 비어 있다. 현재 KakaoProvider({ clientId, clientSecret })는 authorization.params.scope를 넘기지 않는다. 닉네임, 사진, 이메일은 카카오 앱에 이미 허용된 항목만 user/me에 온다.

구글 공급자(node_modules/next-auth/providers/google.js)는 wellKnown으로 https://accounts.google.com/.well-known/openid-configuration을 읽고, 인가 파라미터에 scope: "openid email profile"을 넣는다. checks는 pkce와 state다.

둘 다 clientId와 clientSecret을 process.env.KAKAO_CLIENT_ID, process.env.KAKAO_CLIENT_SECRET, process.env.GOOGLE_CLIENT_ID, process.env.GOOGLE_CLIENT_SECRET에서 읽는다. 값이 없으면 빈 문자열이다. secret은 process.env.NEXTAUTH_SECRET이다.

세션 전략은 jwt이고 maxAge는 60 * 60 * 2초다. pages.signIn은 "/login"이다.

참고: NextAuth Kakao https://next-auth.js.org/providers/kakao, NextAuth Google https://next-auth.js.org/providers/google, NextAuth OAuth https://next-auth.js.org/configuration/providers/oauth

3. 브라우저에서 콜백까지

signIn("google")과 signIn("kakao")는 두 번째 인자가 없다. next-auth/react의 기본은 페이지를 그 공급자의 시작 URL로 옮기는 것이다. LoginForm의 signIn("credentials", { redirect: false })만 응답 객체를 페이지에 남긴다.

OAuth 순서는 next-auth 4의 core/routes/callback.js가 provider.type === "oauth"일 때 처리한다.

1. 버튼 클릭이 signIn("google") 또는 signIn("kakao")를 호출한다.
2. 브라우저가 /api/auth/signin/{id}와 인가 URL로 이동한다.
3. 공급자가 브라우저를 /api/auth/callback/{id}로 되돌린다. 이 경로는 app/api/auth/[...nextauth]/route.ts의 GET 핸들러다.
4. 패키지가 code를 토큰으로 바꾸고 사용자 정보를 읽는다. profile은 공급자 profile()이 고른 사용자다. OAuthProfile은 공급자가 준 원본이다.
5. 전략이 jwt이면 기본 토큰을 만든 뒤 앱의 jwt 콜백을 호출한다.

기본 토큰은 콜백보다 먼저 만들어진다.

```
{
  name: user.name,
  email: user.email,
  picture: user.image,
  sub: user.id?.toString()
}
```

jwt 콜백 인자는 { token: defaultToken, user, account, profile: OAuthProfile, isNewUser, trigger }다. profile은 매핑된 user가 아니라 원본 OAuthProfile이다. account에는 provider, providerAccountId, access_token, expires_at 같은 토큰 응답이 있다.

콜백이 돌려준 객체는 JWT로 인코드되어 세션 쿠키가 된다. 이후 요청은 쿠키를 디코드한 토큰만 `jwt` 콜백에 넘긴다. 그때는 `account`, `profile`, `user`가 없다. 카카오와 구글의 `console.log`는 로그인 직후 한 번에만 찍힌다.

참고: NextAuth signIn https://next-auth.js.org/getting-started/client#signin, NextAuth jwt callback https://next-auth.js.org/configuration/callbacks#jwt-callback, App Router의 NextAuth https://next-auth.js.org/configuration/nextjs#in-app-router

4. 현재 jwt 콜백이 세션에 남기는 값

`session` 콜백은 토큰을 아래 모양으로 다시 만든다. `types/next-auth.d.ts`의 `Session`과 같다.

세션 필드	토큰에서 읽는 곳
`session.accessToken`	`token.accessToken`. 없으면 `""`
`session.expiresIn`	`token.expiresIn`. 없으면 `60 * 60 * 2`
`session.user.id`	`token.id`. 없으면 `""`
`session.user.email`	`token.email`. 없으면 `""`
`session.user.name`	`token.name`. 없으면 `""`
`session.user.profileImageUrl`	`token.profileImageUrl`

`token.picture`와 `token.sub`는 기본 토큰에 있어도 이 콜백이 세션 사용자로 복사하지 않는다.

카카오

`account.provider === "kakao"`이면 원본 `profile`, `account`, 매핑된 `user`를 로그한 뒤 토큰에 넣고 `return`한다. 아래 `if (user)`에는 들어가지 않는다.

토큰	현재 대입
`accessToken`	`account.access_token`
`expiresIn`	`account.expires_at`이 숫자일 때만 `expires_at - 지금(초)`
`id`	`user.id`, 없으면 원본 `profile.id` 또는 `account.providerAccountId`
`email`	`user.email`, 없으면 `profile.kakao_account.email`, 없으면 기존 토큰
`name`	`user.name`, 없으면 `profile.kakao_account.profile.nickname`
`profileImageUrl`	`user.image`, 없으면 `profile.kakao_account.profile.profile_image_url`

패키지 `profile()`도 같은 카카오 필드에서 `id`, `name`, `email`, `image`를 만든다. 원본에 그 필드가 없으면 세션 문자열은 빈 값이 될 수 있다.

구글

`account.provider === "google"`이면 원본 `profile`, `account`, `user`를 로그하고 `token.accessToken`과 `token.expiresIn`만 넣는다. `return`이 없다.

그 다음 `if (user)`가 로그인 직후에는 참이다. 이 블록이 구글에서 방금 넣은 값을 덮는다.

- `token.accessToken = user.accessToken`. 구글 `profile()`은 `accessToken`을 만들지 않으므로 이 값은 `undefined`다.
- `token.expiresIn = user.expiresIn`. 마찬가지로 `undefined`다.
- `token.id = user.id`. 구글 `profile()`의 `id`는 `profile.sub`다.
- `user.profileImageUrl`이 있을 때만 `token.profileImageUrl`을 쓴다. 구글이 채우는 필드 이름은 `image`이고, 기본 토큰은 그것을 `picture`에 넣는다. 현재 세션 사용자는 `picture`를 읽지 않는다.

그래서 구글 로그인 직후 `session` 콜백이 만드는 객체는 `accessToken`이 `""`이고, `expiresIn`은 기본 2시간이며, `user.profileImageUrl`은 없다. `user.email`과 `user.name`은 기본 토큰에 이미 있으므로 원본에 있었다면 세션에 남는다.

클라이언트 `AuthProvider`는 `accessToken`만 받는다. `isLoggedIn`은 `Boolean(accessToken)`이다. 빈 문자열은 로그아웃으로 보인다. `app/(auth)/login/page.tsx`의 `readAccessToken()`도 같은 쿠키 필드를 보고, 문자열이 비어 있으면 `null`이라 `/`로 보내지 않는다.

자격 증명

`authorize`가 돌려주는 사용자에는 `accessToken`과 `expiresIn`이 있다. OAuth 분기를 타지 않고 `if (user)`만 실행되므로 그 두 값이 토큰에 남는다.

5. 페이지가 세션을 읽는 곳

`app/layout.tsx`는 `<Suspense fallback={null}>` 안에 `AuthShell`을 둔다.

`AuthShell`은 `await connection()` 다음에 `getServerSession(authOptions)`을 호출한다. `connection()`은 들어오는 요청까지 기다린다. 이 호출이 없으면 루트 레이아웃을 미리 그릴 때 `getServerSession` 안의 `crypto.randomBytes`가 렌더마다 바뀌는 값으로 막힌다. 세션 객체 전체는 서버 로그에 찍고, 클라이언트로 넘기는 값은 `session.accessToken`뿐이다.

이 트리에는 `SessionProvider`와 `useSession`이 없다.

로그 제목	함수	언제
`kakao profile`, `kakao account`, `kakao user`	`jwt`	카카오 콜백 직후. 이후 페이지 요청에는 다시 안 나온다
`google profile`, `google account`, `google user`	`jwt`	구글 콜백 직후
`session`	`session`	`getServerSession`이 세션을 만들 때마다
`getServerSession`, `AuthShell`	`AuthShell`	레이아웃이 세션을 읽을 때마다
`AuthProvider`	클라이언트 컴포넌트	브라우저 콘솔. 값은 `{ accessToken, isLoggedIn }`뿐이다

이미 발급된 JWT에는 이번 콜백이 소급되지 않는다. 공급자 로그를 보려면 그 공급자로 다시 로그인해야 한다. 카카오 버튼은 현재 화면에 없다.

참고: NextAuth session callback https://next-auth.js.org/configuration/callbacks#session-callback, NextAuth getServerSession https://next-auth.js.org/configuration/nextjs#getserversession, Next.js connection https://nextjs.org/docs/app/api-reference/functions/connection

6. redirect 콜백

`authOptions.callbacks.redirect`는 세 갈래다.

- `url`이 `/`로 시작하면 `${baseUrl}${url}`을 돌려준다.
- `url`의 origin이 `baseUrl`과 같으면 그 `url`을 돌려준다.
- 그 외에는 `baseUrl`만 돌려준다.

OAuth가 끝난 뒤 이동도 이 함수를 탄다. 다른 사이트로 보내지 않는다.

참고: NextAuth redirect callback https://next-auth.js.org/configuration/callbacks#redirect-callback